

# Document de conception — PertFlow

Ce document décrit l'**architecture** de PertFlow, ses **choix techniques** et leurs **justifications**. Il s'adresse à un développeur qui doit comprendre ou faire évoluer l'outil. Le compagnon maintenance.md détaille la reprise pratique et les pièges.

## 1. Objectif et contraintes structurantes

PertFlow est un outil de planification **PERT** en **application web locale** (un fichier HTML). Deux contraintes dictent toute l'architecture :

1. **Ouverture en `file://` par double-clic — PRIMORDIAL.** L'outil tourne sur des postes d'entreprise verrouillés par la DSI : **aucun serveur, aucun build, aucune architecture client-serveur**. Deux conséquences absolues : - **Pas de modules ES6** ( `<script type="module">` + `import / export` ) : ils forceraient un serveur (CORS en `file://` ). Tout le code est chargé par des `<script src>` **classiques**, et vit dans le **scope global** (les fonctions `function pertX()` sont globales). - **Pas de `fetch()` /XHR de fichiers locaux** : les fichiers utilisateur sont lus via `<input type="file">` + `FileReader` , jamais par `fetch` .
2. **Licence MIT uniquement** : toutes les bibliothèques sont locales ( `lib/` ) et sous licence permissive compatible MIT. Aucune dépendance réseau au runtime.

## 2. Vue d'ensemble et pile technique

| Besoin                            | Choix                                | Licence | Fichier                                                |
|-----------------------------------|--------------------------------------|---------|--------------------------------------------------------|
| Canvas / graphe                   | <b>LiteGraph.js</b> (build « core ») | MIT     | <code>lib/litegraph.core.js</code> + <code>.css</code> |
| Export PDF                        | <b>jsPDF</b>                         | MIT     | <code>lib/jspdf.umd.min.js</code>                      |
| Zip/dézip (import & export Excel) | <b>fflate</b>                        | MIT     | <code>lib/fflate.min.js</code>                         |
| Export PNG                        | API Canvas2D native                  | —       | —                                                      |
| UI                                | Vanilla JS + CSS custom              | —       | —                                                      |

Aucun framework (React/Vue...), aucun bundler au sens classique : la simplicité prime, et la contrainte `file://` l'impose.

**LiteGraph : le build « core », jamais le build complet.** L'éditeur publie deux fichiers.

`build/litegraph.js` embarque, en plus du canevas, une bibliothèque de ~156 types de nœuds (audio, MIDI, webcam, `network/websocket` , `network/httprequest` ...) dont PertFlow n'utilise rien — il définit ses propres nœuds. Or **ouvrir un `.pert` instancie tout type déclaré dans le fichier** : un planning forgé y posait un nœud WebSocket et ouvrait une connexion sortante à l'ouverture (audit du 01/09/2026, constat C-01 — reproduit et corrigé). `litegraph.core.js` ne contient aucun de ces nœuds : un type inconnu devient un nœud vide, sans comportement, et le planning s'ouvre normalement. **Ne pas revenir au build complet** ; `tools/smoke-securite.js` refuse la suite si cela arrive.

## Structure des fichiers

```
pertflow/
├─ index.html          # Point d'entrée : DOM + <script src> de tout le code
├─ lib/                # Bibliothèques locales (LiteGraph « core », jsPDF, fflate)
│   └─ LICENCES-TIERCES.txt # Licences MIT des 3 libs – part dans l'archive de livraison
├─ css/style.css       # Styles globaux (thème sombre)
├─ src/
│   ├─ nodes.js        # Types de nœuds PERT + rendu custom (LiteGraph)
│   ├─ pert_engine.js  # Calcul PERT, conversions dates↔unités, layout, chemin critique
│   │                  #   estimation charge/coût
│   ├─ ui.js           # Toolbar, panneau, dialogues, menus, filtre, barre de statut
│   ├─ storage.js      # Sérialisation/chargement .pert
│   ├─ history.js      # Undo/Redo par snapshots
│   ├─ autosave.js    # Filet anti-crash (snapshot localStorage)
│   ├─ align.js        # Boîte d'alignement / distribution des nœuds sélectionnés
│   ├─ t0_marker.js    # Repère T0 + bande « travaux anticipés » (2 couches de rendu)
│   ├─ time_grid.js    # Trame calendaire de fond, optionnelle et d'intensité réglable
│   ├─ synthesis.js    # Fenêtre de synthèse (planification) + helpers de rendu personnalisés
│   ├─ suivi.js        # Fenêtre de suivi d'avancement – réutilise ces helpers
│   ├─ import.js       # Fenêtre d'import + registre des formats (même patron que l'export)
│   ├─ import_excel.js # Import des .xlsm legacy CPERT (fflate + DrawingML)
│   ├─ import_pert.js  # Import/concaténation d'un .pert dans le planning courant
│   ├─ export.js       # Fenêtre d'export + PNG/PDF + helpers de téléchargement
│   ├─ export_csv.js   # Export CSV
│   ├─ export_xlsx.js  # Mini-writer XLSX générique (sur fflate)
│   ├─ export_gantt.js # Gantt chargé (Excel) + MS Project (MSPDI XML)
│   ├─ export_microjalons.js # Micro-jalonnement (Excel)
│   ├─ export_planning_directeur.js # Planning directeur (Excel) : canevas calendrier
│   └─ link_routing.js # Rendu des liens : styles + routage orthogonal (évitement)
├─ docs/               # Manuel, conception, maintenance, notes de version (MD + HTML)
├─ tools/              # Suite de tests + captures d'écran (cf. tools/README.md)
├─ test_cases/         # Jeux d'essai – versionnés en LISTE BLANCHE (cf. tools/README.md)
├─ scripts/
│   ├─ build-bundle.js # Génère le fichier autonome dist/pertflow.html
│   └─ make-release.js # Fabrique l'archive de livraison (bundle + manuel + notes)
└─ dist/
    ├─ pertflow.html   # Livrable autonome (versionné)
    ├─ release/        # Archives de livraison – GITIGNORÉ (hébergées par GitHub Releases)
    └─ audit/          # Rapports d'audit de sécurité – GITIGNORÉ (cf. §9)
```

L'**ordre de chargement** des `<script>` dans `index.html` matérialise les dépendances (chaque module s'appuie sur les globales définies avant lui ; `ui.js` est chargé en dernier et fait le câblage).

---

## 3. Modèle de données

---

### Le fichier `.pert`

```
{
  "version": "1.0",
  "meta": { "title", "t0", "unit", "layout_gap", "link_mode", "prop_width",
            "hours_per_month", "hours_per_day", "hourly_rate", "groups", "autosave" }
  "graph": { /* sérialisation LiteGraph native (graph.serialize()) */ }
}
```

- `meta.unit`  $\in$  `"j" | "sem" | "mois"`.
- `meta.groups` = registre `{ nom_de_groupe: couleur }` (mémoire des couleurs de groupe).
- Les **valeurs calculées** (ES/EF/LS/LF/slack/is\_critical, coût) **ne sont PAS sérialisées** : elles sont **recalculées** au chargement (`pertRecalc`) → cohérence garantie même sur un vieux fichier. Les propriétés **saisies** vivent dans `node.properties` (sérialisées nativement par LiteGraph).

### Les nœuds

- **Activité** (`pert/activity`) : `uid`, `label`, `duration`, `etp`, `charge_mode`, `charge_hours`, `responsable`, `notes`, `group`, `color`.
  - `charge_mode` (`"etp"` par défaut | `"heures"`) désigne **laquelle des deux expressions de la charge est saisie** ; l'autre est déduite via l'élongation et les paramètres de coût. Ce drapeau n'est pas cosmétique : il dit ce qui reste **invariant** quand la durée change (en `"etp"` la charge horaire suit la durée, en `"heures"` c'est l'ETP déduit qui se dilue). Stocker les deux valeurs sans lui les ferait diverger dès la première modification de durée.
  - Ne **jamais** lire `etp` / `charge_hours` en direct : passer par `pertActivityEtp` / `pertActivityHours` (`pert_engine.js`), seuls à connaître le mode. `pertActivityEtp` rend `null` quand l'ETP n'existe pas (charge en heures sur une durée nulle).
  - Un `.pert` antérieur ne porte ni `charge_mode` ni `charge_hours` : les défauts du constructeur subsistent (LiteGraph **fusionne** les propriétés sérialisées sur celles du nœud neuf) → **aucune migration**.
  - **Jalon** (`pert/milestone`) : `label`, `due_date`, `tag` (`"` | `DOTD` | `COTD` | `ING`).
  - **Label** (`pert/label`) : `text` (aucun lien, hors calcul).
-

## 4. Le moteur PERT ( `pert_engine.js` )

---

### Principe

- **Calcul interne en unités** (offsets depuis T0), **conversion en dates** à l'affichage. On convertit toujours l'**offset cumulé** (jamais pas-à-pas) → conversions inversibles, sans dérive.
- **Mois = mois calendaires réels** ( `pertAddUnits` via `Date.setMonth` ), pas un facteur fixe de 30 jours (qui dérivait sur les projets longs). Jours ( $\times 1$ ) et semaines ( $\times 7$ ) sont exacts.
- **Forward pass** :  $ES = \max(EF \text{ des prédécesseurs})$  ,  $EF = ES + \text{durée}$  .
- **Backward pass** :  $LF = \min(LS \text{ des successeurs})$  ,  $LS = LF - \text{durée}$  . **Ancre** =  $\max(EF)$  (fin de projet).
- **Marge**  $\text{slack} = LF - EF$  . **Chemin critique** = marge **minimale** (  $\text{slack} \leq \text{minSlack} + \epsilon$  ), ce qui reste correct même quand une échéance imposée rend des marges **négatives**.
- **Détection de cycle** par DFS tricolore avant tout calcul.

### Subtilité des jalons

Le rôle d'un jalon dépend de sa **topologie** (calculé dans le forward pass) :

- **Jalon entrant** (aucun entrant + au moins un sortant + `due_date` ) :  $ES = EF = \text{offset}(\text{due\_date})$  (planché à T0). Modélise une **contrainte externe** qui retarde la chaîne aval.
- **Jalon terminal / point de contrôle** : la `due_date` **borne le LF**, ne force pas l'ES.
- Le drapeau `target_missed` (EF calculé > cible) pilote l'alerte visuelle.

### Rendu du chemin critique

`pertHighlightCriticalPath` colore les **liens** contraignants (via `link.color` , mécanisme natif LiteGraph) et mémorise l'ensemble des nœuds réellement mis en évidence dans `window.pertCriticalPathIds` — dont dérive la barre de statut (coût + nombre), pour rester cohérent avec le tracé rouge, qu'il y ait ou non une sélection.

### Layout automatique

`pertAutoLayout` : packing par couloirs, abscisse  $\propto$  ES (allure Gantt), tâches d'un même groupe sur des couloirs voisins, jalons terminaux en bande haute. **Déclenché manuellement** uniquement (bouton « Réorganiser ») pour ne jamais casser un placement à la main.

---

## 5. Rendu LiteGraph custom ( `nodes.js` , `link_routing.js` )

---

LiteGraph fournit le canvas, le pan/zoom, la sélection, la sérialisation et les liens. Tout le **rendu des nœuds** est **custom** pour coller au PERT.

- **Nœuds dessinés à la main** via `onDrawForeground` / `onDrawBackground` (en-tête coloré multi-lignes, sections calculées, pastilles de tag, coin drapeau des jalons, voile d'estompage du filtre).
- **Masquage de la barre de titre** : `Constructor.title_mode = LiteGraph.NO_TITLE` (⚠ le flag d'instance `flags.no_title` **n'a aucun effet** — piège classique).
- **Positions de slots explicites** ( `input.pos` / `output.pos` ) puisque le titre est masqué.
- **Filtre** : voile translucide **sombre** dessiné en `onDrawForeground` (donc par-dessus le contenu et les slots). L'état de filtre `window.pertFilter` est un **état de vue non sérialisé**. Quatre natures : groupe, couleur, responsable — qui ne concernent que les Activités — et **recherche** ( `type:"text"` , v0.20), la seule qui porte sur les **trois types de nœuds**, sur leur nom comme sur leurs notes, insensible à la casse et aux accents.
- **Rendu des liens** : `renderLink` est **surchargé sur l'instance** `LGraphCanvas` (sans patcher la lib). Trois styles ( `meta.link_mode` ) : courbe (spline natif), droit (straight natif), et **coudé** = routage **orthogonal custom** qui **contourne** les nœuds (best-effort : canal vertical, sinon bande horizontale, sinon tracé direct). Garde-fous perf : élagage spatial + dégradation au-delà de 300 nœuds. Le lien élastique de création reste une courbe native.

## 6. UI, historique et récupération ( `ui.js` , `history.js` , `autosave.js` )

---

- **Menus contextuels** : `getMenuOptions` / `getNodeMenuOptions` **surchargés** sur l'instance pour remplacer les menus natifs anglais par des menus français recentrés PERT ; searchbox native neutralisée.
- **Panneau Propriétés** : reconstruit à la sélection ; helpers `buildField` , `buildCombobox` (menu déroulant custom à pastilles, fiable multi-navigateurs — le `<datalist>` natif est inadapté), `buildSelect` , `buildTextarea` , `buildReadonly` . Depuis v0.19, il est scindé en **deux onglets** — *Propriétés* (saisie) et *Synthèse* (valeurs calculées + prédécesseurs / successeurs) — sur la ligne de partage « saisi / déduit ». Le panneau inactif est **masqué, jamais retiré du DOM** (les fonctions de rafraîchissement retrouvent leurs conteneurs par id), le bouton **Supprimer** vit dans un pied fixe hors de la zone qui défile, et l'onglet consulté est mémorisé entre deux sélections. Le voisinage est lu via `pertBuildAdjacency` (moteur) : une seule source de vérité avec le calcul PERT.
- **Undo/Redo** : historique par **snapshots** ( `meta + graph.serialize()` ), restaurés par `configure()` — même mécanisme que la persistance, donc exhaustif. Coalescence des frappes.
- **Sauvegarde automatique** : en `file://` , impossible d'écrire un fichier silencieusement → un **snapshot de récupération dans `localStorage`** , écrit périodiquement tant qu'il reste du travail

non sauvegardé, proposé à la restauration au démarrage. Activée par défaut.

- **Gestion d'erreurs** : `showToast` / `showError` / `guardUI` + filet global — indispensable en `file://` où l'utilisateur n'a pas la console.

## Filtre et recherche ( `window.pertFilter` )

Cinq natures de filtre — `group`, `color`, `responsable`, `progress` (qui ne « matchent » que des Activités) et `text` (nom **et** notes, sur les trois types de nœuds, insensible à la casse et aux accents). Un filtre **n'enlève rien** : il **estompe** les nœuds hors sélection sous un voile, pour que le planning reste lisible dans son ensemble.

C'est un **état de vue** : jamais sérialisé dans le `.pert`, au même titre que l'onglet courant du panneau ou de la synthèse. Une recherche **remplace** le filtre courant (ils partagent une seule variable), d'où l'obligation de resynchroniser les marqueurs d'état de la liste — sans quoi l'IHM annonce un filtre qui n'est plus actif.

## Fenêtres de rapport ( `synthesis.js`, `suivi.js` )

Le bouton **Synthèse** ouvre un menu à deux entrées, deux fenêtres au même patron :

- **Synthèse de planification** — le planning **tel qu'il est prévu** : vue d'ensemble, jalons entrants / sortants, agrégats par groupe (charge, coût, fin au plus tard), et un onglet **Analyse**. Ses quatre onglets deviennent **quatre chapitres** à l'impression.
- **Suivi d'avancement** — le même planning **confronté à la date du jour**. Il ne recalcule rien. La date du point est surchargeable ( `window.pertSuiviToday` ) : sans cette couture, ni test ni capture ne seraient reproductibles.

Deux principes structurants. **L'onglet Analyse est fait pour s'enrichir** : un contrôle est un objet `{ id, title, hint, columns, rows }` poussé dans une liste, le rendu est générique ; un contrôle **sans anomalie n'est pas affiché** (une liste de choses à regarder, pas des cases vertes à faire défiler) et chaque ligne doit **mener au(x) nœud(s)** concerné(s). **La coquille de fenêtre est mutualisée** par des **classes** ( `.synth-content`, `.synth-tabs`, `.synth-printing` ) et non par les ids de la synthèse — deux fenêtres coexistant, une règle `@media print` qui ciblerait un id force-afficherait la fenêtre fermée sur le papier.

---

## 7. Import Excel legacy ( `import_excel.js` )

Le `.xlsm` est un ZIP dont **toute la donnée utile est dans** `xl/drawings/` (les groupes de formes = nœuds, les connecteurs = liens ; les cellules sont cosmétiques sauf l'onglet **MANUEL** de configuration : feuille cible, T0, unité). Traitement 100 % `file://` :

- **Dézip par fflate** ( `unzipSync` ), parsing **DOMParser** natif, `<input type="file">` + `FileReader.readAsArrayBuffer` , **jamais fetch** .
- Convention de nommage : `A` =activité, `S` =jalón, `E` =**jalón d'entrée** (matérialisé en jalón entrant avec ses arêtes). La couche **transforms purs** ( `buildImportModel` ) est séparée de la couche DOM/ZIP → testable en Node.

## 8. Exports ( `export*.js` )

Un **seul bouton** ouvre une fenêtre listant les formats (liste data-driven `PERT_EXPORT_FORMATS` + `pertRegisterExportFormat` , triée par `order` ).

- **PNG / PDF** : **rendu hors-écran** indépendant du zoom (un `LGraphCanvas` temporaire calé sur la boîte englobante, fond blanc, un seul `draw` ), puis `toDataURL` / `jsPDF` (A4, `compress:true` ).
- **CSV** : dump brut, séparateur `;` , décimales `,` , BOM UTF-8.
- **XLSX** : un `.xlsx` est un **ZIP de XML** → **mini-writer maison sur fflate** ( `export_xlsx.js` , `pertXlsxBuild` ) : cellules texte/nombre/date/formule, styles (formats date, `0.00` , fills, polices, bordures, alignements) dédoublés, `sharedStrings` , hauteurs de ligne, cellules fusionnées, panneaux figés, mise en page — et **formes flottantes DrawingML** ( `shapes` , ancrage `twoCellAnchor` ). **Pas de SheetJS** (Apache-2.0, exclu par « MIT uniquement »). Tout ce qui a été ajouté en v0.23 est **optionnel** : un appelant qui ne passe que `{ name, cols, rows }` produit le classeur d'avant, octet pour octet.

L'ordre des éléments d'une `<worksheet>` est **imposé par le schéma OOXML** ( `sheetPr` , `sheetViews` , `sheetFormatPr` , `cols` , `sheetData` , `mergeCells` , `printOptions` , `pageMargins` , `pageSetup` , `drawing` ). Excel refuse d'ouvrir un classeur qui s'en écarte **sans dire lequel** est en cause : la séquence est écrite une fois pour toutes dans `pertXlsxBuild` , ne pas y insérer un élément « au plus commode ».

- **Gantt chargé** et **MS Project (MSPDI XML)** partagent `pertScheduleModel()` (tri groupes par ES précoce, classement jalons entrée/sortie, colonnes de périodes, liens). Aucune bibliothèque `.mpp` native n'existant côté navigateur (MIT/offline), MS Project est produit en **MSPDI XML** écrit à la main.



**MS Project REPLANIFIE à l'import** — il ne relit pas les dates fournies. Une tâche auto-planifiée est « dès que possible » : le `<Start>` d'une tâche **sans prédécesseur** est écrasé par le début du projet (c'est ainsi que les jalons d'entrée s'empilaient sur T0), et celui d'une tâche **avec** prédécesseurs est de toute façon recalculé. D'où les deux règles de v0.23.2 : une date ne « tient » qu'accompagnée d'une **contrainte** (SNET) — posée sur les tâches sans prédécesseur et sur les tâches anticipées, les seules dont l'amont ne dicte pas la date — et la cible d'un jalon de **sortie** part dans `<Deadline>`, pas dans `<Start>` (celle d'un jalon d'**entrée** reste une donnée d'entrée : aucune deadline, cf. son LF non borné). Corollaire : **MSPDI ne transporte aucune mise en forme** (la couleur des barres vit dans le `.mpp`) → groupe et couleur partent en **champs personnalisés** `ExtendedAttribute` (`Texte1/Texte2`), déclarés **avant** `<Tasks>` — l'ordre des éléments y est imposé comme en OOXML.

- **Planning directeur** ( `export_planning_directeur.js`, v0.23) : transposition du PERT sur un canevas calendaire (une colonne = un mois, en-tête années/trimestres/mois, zébrage, panneaux figés, paysage). Trois règles de cadrage le gouvernent, décidées avec l'utilisateur et à ne pas défaire sans nouvel arbitrage : 1. **Abscisse = les dates** ( $ES \rightarrow EF$ , cible pour un jalon) ; **ordonnée = le canvas** ( `pos[1]` du nœud, à l'échelle près). Aucun regroupement, aucun tri, aucun ré-empilement. Le `x` du canvas ne sert qu'aux **Labels**, qui n'ont pas de date : leur abscisse passe par une régression  $x \rightarrow \text{position en mois}$  ajustée sur les nœuds datés ( `pertPdLabelXFit` ). 2. Les **colonnes B et C restent vides**, mises en forme, à remplir par l'utilisateur : le canevas a deux niveaux de nomenclature là où PertFlow n'a que le groupe. 3. **Aucun lien tracé** ; **légende** des couleurs en haut à gauche.

Les **bandes** ( `bands` ) sont une *lecture* de l'agencement, pas une réorganisation : les nœuds dont les boîtes se chevauchent verticalement partagent une ligne de classeur, le vide entre deux bandes est partagé par moitié. Le plafond `PERT_PD_MAX_GAP_PT` ne borne que **cette marge** — plafonner la hauteur totale comprimerait le contenu et ferait se recouvrir deux nœuds distincts d'une même bande.

**Pourquoi les barres ne sont pas des cellules colorées** (contrairement au Gantt chargé) : une tâche démarre au milieu d'un mois. La coloration de cellule oblige à arrondir à la colonne ; l'ancrage `twoCellAnchor` place la forme à (colonne, décalage dans la colonne) et rend la date exacte. Corollaire précieux : une mauvaise estimation de la largeur rendue d'une colonne ne fausse que la fraction interne, elle **ne se cumule pas** le long de la grille. - Téléchargements via `pertDownloadBlob` (objet URL, fonctionne en `file://` ).

## 9. Packaging ( scripts/build-bundle.js )

Le développement se fait sur la structure `index.html + src/ + lib/`. Un **script Node natif sans dépendance** produit le **livrable autonome** `dist/pertflow.html` en **inlinant** les `<link>` / `<script>` (avec garde-fou : échec s'il reste une référence `lib/ / src/ / css/`). Il injecte `window.PERTFLOW_BUILD = { date, tag }` (lu par la popup « À propos »). Le bundle est **versionné** et régénéré en fin de session.

### Ce que le bundle porte en plus des sources

Deux choses n'existent **que** dans le bundle, injectées au build — c'est délibéré, car c'est le bundle qui circule :

| Injection                                                                                                      | Pourquoi pas dans <code>index.html</code>                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Content-Security-Policy</b><br>( <code>default-src 'none'</code> ,<br><code>connect-src 'none'</code> ... ) | En développement les scripts sont des fichiers séparés chargés en <code>file://</code> , que <code>script-src 'unsafe-inline'</code> bloquerait : l'application ne démarrerait plus. Dans le bundle tout est en ligne. |
| <b>Crédits et licences</b> des trois bibliothèques                                                             | La licence MIT impose de faire voyager la mention de copyright avec les copies ; c'est le fichier distribué qui est une copie.                                                                                         |

La CSP est la **seconde barrière** du constat C-01 (cf. §2) : le build « core » supprime le code fautif, `connect-src 'none'` en interdit l'effet — y compris pour du code à venir. Les deux sont indépendantes à dessein. Une CSP échoue en **silence** côté utilisateur (fonction morte, aucune erreur visible) : le contrôle qui l'attrape est `tools/smoke-securite.js` , qui produit un export réel et exige zéro violation.

Les deux injections sont ancrées sur le `<meta charset>` d' `index.html` . Un remaniement du `<head>` les rendrait sans effet **sans rien casser** — d'où un garde-fou qui **refuse d'écrire le bundle** si l'ancre a disparu (comme les deux refus de `make-release.js` ).

### Audit de sécurité rejouable ( tools/audit-securite.js )

`node tools/audit-securite.js` refait, sur le fichier du jour, les contrôles du dossier remis à une DSI : empreintes, **provenance comparée aux paquets npm officiels** (la version de chaque bibliothèque est *déduite de son empreinte*, jamais lue dans un numéro déclaré), vulnérabilités connues (base OSV), inventaire des API à effet de bord, reproductibilité du bundle depuis ses sources, et **comportement observé** dans un vrai navigateur (planning forgé + parcours complet). Sortie : `dist/audit/audit-securite-<tag>.html` et `.pdf` , **gitignorés** et absents de l'archive — document de travail daté, remis sur demande, pas une certification qui accompagnerait le produit.

Deux règles y sont structurantes : **les contrôles réseau ne font jamais échouer l'audit** (un poste verrouillé n'a pas accès à npm — ils rendent « non vérifié », et le rapport le dit), et **aucun attendu n'est**

**codé en dur** (un attendu recopié d'une version antérieure ferait passer pour vérifié ce qui ne l'est plus).

## 10. Récapitulatif des choix et de leurs raisons

| Choix                                                                  | Raison                                                                                               |
|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Pas de modules ES6, tout en global via <code>&lt;script src&gt;</code> | Ouverture <code>file://</code> sans serveur (CORS)                                                   |
| <code>FileReader</code> au lieu de <code>fetch</code>                  | Lecture de fichiers locaux en <code>file://</code>                                                   |
| fflate pour lire <b>et</b> écrire les Excel                            | MIT, déjà nécessaire à l'import ; évite SheetJS (Apache-2.0)                                         |
| MS Project en MSPDI XML                                                | Aucune lib <code>.mpp</code> native MIT/offline en navigateur                                        |
| Valeurs PERT recalculées, non sérialisées                              | Cohérence garantie au chargement                                                                     |
| Rendu de nœuds/liens custom sur l'instance                             | Coller au PERT sans patcher LiteGraph                                                                |
| Snapshots pour l'undo <b>et</b> l'autosave                             | Un seul mécanisme robuste et exhaustif                                                               |
| Autosave en <code>localStorage</code>                                  | Seul stockage persistant possible en <code>file://</code>                                            |
| Layout manuel (jamais auto)                                            | Ne jamais casser un placement à la main                                                              |
| Filtre et onglets = <b>état de vue</b> , hors <code>.pert</code>       | Un fichier décrit un planning, pas une session de travail                                            |
| Avancement <b>hors de tout calcul</b>                                  | Le PERT reste l'objectif : le renseigner ne doit rien déplacer                                       |
| Charge : un <b>mode</b> de saisie, pas deux valeurs libres             | Le mode dit l'invariant quand la durée bouge ; deux valeurs stockées sans lui divergeraient          |
| Filtre qui <b>estompe</b> au lieu de masquer                           | Garder le planning lisible dans son ensemble                                                         |
| LiteGraph en build « <b>core</b> »                                     | Un <code>.pert</code> ne doit pouvoir instancier que les 3 types de PertFlow                         |
| CSP injectée <b>au build</b> , pas dans les sources                    | Le bundle est ce qui circule ; en <code>file://</code> la même règle casserait le mode développement |